Pseudocode / Python code of popular SORTING ALGORITHMS:

I have used several resources: GeekForGeeks, W3schools, TutorialsPoint, Programiz **;** also Gemini, GPT4.0, Perplex AI, Copilot (the python code was generated by Bing Copilot).

Sorting can be **comparison-based** or **non-comparison-based.**

Comparison-based Sorting Algorithms :

Bubble Sort

Insertion Sort

Selection Sort

Merge Sort

Quick Sort

Heap Sort

Tim Sort

Non-comparison based Sorting Algorithms :

Counting Sort

Radix Sort

Bucket Sort

Bitonic Sort

The following information was provided by Copilot.

Comparison-Based Sorting Algorithms

Algorithm	Stable	Description
Bubble Sort	Yes	Bubble Sort is one of the simplest sorting algorithms. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted. Although it is easy to understand and implement, Bubble Sort is inefficient for large datasets due to its quadratic time complexity. It is mainly used for educational purposes.
Insertion Sort	Yes	Insertion Sort builds the final sorted array one element at a time. It takes each element and places it in its correct position by comparing it with the elements in the already sorted part of the array. This algorithm is efficient for small datasets or partially sorted arrays. It is simple and has a good performance on nearly sorted data.
Selection Sort	No	Selection Sort divides the input list into two parts: a sorted sublist of items that is built up from left to right at the front (left) of the list and a sublist of the remaining unsorted items. The algorithm repeatedly selects the smallest (or largest) element from the unsorted sublist, swaps it with the leftmost unsorted element, and moves the sublist boundaries one element to the right. It is not stable and has a quadratic time complexity, making it inefficient for large datasets.
Merge Sort	Yes	Merge Sort is a divide-and-conquer algorithm that divides the input array into two halves, recursively sorts each half, and then merges the two sorted halves. It is stable and has a consistent time complexity of O(n log n) across all cases, making it suitable for large datasets. However, it requires additional space for the temporary arrays used during merging.
Quick Sort	No	Quick Sort is another divide-and-conquer algorithm. It selects a 'pivot' element, partitions the array into two halves such that elements less than the pivot are on the left and elements greater than the pivot are on the right, and then recursively sorts the two halves. Quick Sort is very efficient on average but has a worst-case time complexity of $O(n^2)$. It is widely used due to its in-place sorting nature and good performance on average.
Heap Sort	No	Heap Sort converts the input array into a binary heap data structure and then repeatedly extracts the maximum element from the heap and rebuilds the heap until all elements are sorted. It is not stable but has a time complexity of O(n log n), making it efficient for large datasets. Heap Sort is an in-place algorithm, meaning it requires only a constant amount of additional space.
Tim Sort	Yes	Tim Sort is a hybrid sorting algorithm derived from merge sort and insertion sort. It is the default sorting algorithm in Python's built-in sort() function and Java's Arrays.sort(). Tim Sort performs exceptionally well on real-world data, which usually contains runs of ordered elements. It is stable and has a time complexity of O(n log n) in the average and worst case.

Cocktail Sort	Yes	Cocktail Sort is a variation of Bubble Sort that sorts in both directions on each pass through the list. This algorithm starts at the beginning of the list, compares and swaps adjacent elements if they are in the wrong order, then reverses direction at the end of the list and repeats the process. It is more efficient than Bubble Sort but still has a quadratic time complexity, making it inefficient for large datasets.
Comb Sort	No	Comb Sort is an improvement over Bubble Sort that compares and swaps elements at a certain gap apart, which decreases progressively in size until it reaches 1. By using larger gaps initially, Comb Sort is able to eliminate the small values present at the end and distribute them throughout the list more quickly. This reduces the total number of comparisons and swaps, making it more efficient than Bubble Sort.
Bogo Sort	No	Bogo Sort is a highly inefficient and mostly impractical sorting algorithm that works by repeatedly shuffling the list and checking if it is sorted. It continues this process until the list is sorted. Due to its random nature, Bogo Sort has an extremely high average time complexity and is only used for educational purposes or as a joke algorithm.
Introsort	No	Introsort, or introspective sort, begins with Quick Sort and switches to Heap Sort when the recursion depth exceeds a level based on the number of elements being sorted. This ensures O(n log n) worst-case performance. Introsort is efficient and practical, combining the fast average performance of Quick Sort with the worst-case efficiency of Heap Sort. It is commonly used in the C++ Standard Library.

Non-Comparison-Based Sorting Algorithms

Algorithm	Stable	Description
Counting Sort	Yes	Counting Sort is an integer sorting algorithm that works by counting the number of objects that have distinct key values (kind of hashing). It uses an auxiliary array to store the count of each unique element and then modifies the count array to place the elements in the correct position. Counting Sort is stable and efficient for sorting integers within a known range, as its time complexity is linear in relation to the number of elements and the range of the input values.
Radix Sort	Yes	Radix Sort is a non-comparison-based sorting algorithm that sorts numbers by processing individual digits. It starts with the least significant digit and moves towards the most significant digit, using a stable sorting algorithm like counting sort as a subroutine to sort the digits. Radix Sort is efficient for large datasets with a fixed range of integers, as its time complexity is linear.
Bucket Sort	Yes	Bucket Sort works by distributing elements into a number of buckets. Each bucket is then sorted individually, either using a different sorting algorithm or by recursively applying the bucket sort. Finally, the sorted buckets are concatenated to form the final sorted array. Bucket Sort is efficient for uniformly distributed data and can achieve linear time complexity in the best case.
Bitonic Sort	No	Bitonic Sort is a parallel sorting algorithm that sorts a bitonic sequence (a sequence that first increases and then decreases, or vice versa). It recursively sorts the sequence and then merges the sorted subsequences. Bitonic Sort is highly parallelizable and is suitable for parallel computing environments. It is not stable and has a time complexity of $O(n \log^2 n)$.

BUBBLE SORT :

BubbleSort(arr):

n = length(arr)

for i = 0 to n – 1:

for j = 0 to n - i - 2:

Swap if the element found is greater than the next element

if arr[j] > arr[j + 1]:

swap(arr[j], arr[j + 1])

```python
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

SELECTION SORT :

It sorts an array by repeatedly selecting the smallest (or largest) element from the unsorted portion and swapping it with the first unsorted element. This process continues until the entire array is sorted.

1. First we find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.
2. Then we find the smallest among remaining elements (or second smallest) and swap it with the second element.
3. We keep doing this until we get all elements moved to correct position

SelectionSort(arr):

n = len(arr)

for i = 0 to n - 1:

 # Assume the minimum is at index i

min_idx = i

 # Find the minimum element in the remaining unsorted array

for j = i + 1 to n - 1:

if arr[j] < arr[min_idx]:

min_idx = j

 # Swap the found minimum element with the element at index i

swap(arr[i], arr[min_idx])

```python
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        # Find the minimum element in remaining unsorted array
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap the found minimum element with the first element
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

INSERTION SORT :

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list

InsertionSort(arr):

n = len(arr)

 # Start from the second element (index 1) since the first element is trivially sorted

for i = 1 to n - 1:

key = arr[i]

j = i – 1

 # Move elements of arr[0..i-1] that are greater than key to one position ahead of their current position

while j >= 0 and arr[j] > key:

arr[j + 1] = arr[j]

j -= 1

 # Place the key in its correct position

arr[j + 1] = key

```
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1

        # Move elements of arr[0..i-1] that are greater than key to one posi
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        # Place the key in its correct position
        arr[j + 1] = key
```

MERGE SORT :

MergeSort(arr):

```
if length of arr > 1:
    # Find the middle point to divide the array into two halves
    mid = length of arr / 2
    # Call MergeSort recursively on the first half
    leftHalf = arr[:mid]
    MergeSort(leftHalf)
    # Call MergeSort recursively on the second half
    rightHalf = arr[mid:]
    MergeSort(rightHalf)
    # Merge the two halves
    i = j = k = 0
    # Copy data to an array
    while i < len(leftHalf) and j < len(rightHalf):
        if leftHalf[i] < rightHalf[j]:
            arr[k] = leftHalf[i]
            i += 1
        else:
            arr[k] = rightHalf[j]
            j += 1
        k += 1

    # Check if any element was left in leftHalf[]
    while i < len(leftHalf):
        arr[k] = leftHalf[i]
        i += 1
        k += 1
```

 # Check if any element was left in rightHalf[]

while j < len(rightHalf):

arr[k] = rightHalf[j]

j += 1

k += 1

Notes :

i: Used to track the current index in the leftHalf array during the merge process.

j: Used to track the current index in the rightHalf array during the merge process.

k: Used to track the position in the original array (arr) where the next element from either leftHalf or rightHalf should be placed.

```python
def merge_sort(arr):
    if len(arr) > 1:
        # Finding the middle of the array
        mid = len(arr) // 2

        # Dividing the array elements into 2 halves
        left_half = arr[:mid]
        right_half = arr[mid:]

        # Recursively sort each half
        merge_sort(left_half)
        merge_sort(right_half)

        # Merging the sorted halves
        i = j = k = 0

        # Copy data to temp arrays left_half[] and right_half[]
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1
```

```python
        # Checking if any element was left in the left_half
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1

        # Checking if any element was left in the right_half
        while j < len(right_half):
            arr[k] = right_half[j]
            j += 1
            k += 1
```

QUICK SORT :

There are mainly three steps in the algorithm:

1. *An array is divided into subarrays by selecting a **pivot element** (element selected from the array).*

   While dividing the array, the pivot element should be positioned in such a way that elements less than pivot are kept on the left side and elements greater than pivot are on the right side of the pivot.

2. *The left and right subarrays are also divided using the same approach. This process continues until each subarray contains a single element.*

3. *At this point, elements are already sorted. Finally, elements are combined to form a sorted array.*

QuickSort(arr, low, high):

```
  if low < high:
    # Partition the array around the pivot and get the pivot index
    pi = Partition(arr, low, high)
    # Recursively sort elements before and after partition
    QuickSort(arr, low, pi - 1)
    QuickSort(arr, pi + 1, high)
```

Partition(arr, low, high):

```
  # Select the pivot element (usually the last element in the array : high)
  pivot = arr[high]
  i = low - 1          # Index of smaller element
# As we iterate through the array, whenever we find an element that is less than or equal to the # # pivot, we increment i and swap the element at i with the current element at j.
  for j = low to high - 1:
    # If current element is smaller than or equal to pivot
    if arr[j] <= pivot:
      i += 1
      swap(arr[i], arr[j])
  # Swap the pivot element with the element at index i+1
  swap(arr[i + 1], arr[high])
  return i + 1
```

HEAP SORT:

A max heap is a binary tree where the value of each parent node is greater than or equal to its children. *Start from the middle of the array (n // 2 - 1) because nodes beyond this index are leaves and already satisfy the heap property. For each node (from the middle to the root), call* ***the heapify function*** *to ensure the subtree rooted at that node satisfies the heap property.*

HeapSort(arr):

```
n = len(arr)

# Step 1: Build a max heap from the input array

for i = n // 2 - 1 to 0:

    heapify(arr, n, i)

# Step 2: Extract elements one by one from the heap

for i = n - 1 to 0:

    # Move the root of the heap (largest element) to the end of the array

    swap(arr[0], arr[i])

    # Call heapify on the reduced heap and ensures that the subtree rooted at index i satisfies the max heap property.

    heapify(arr, i, 0)
```

heapify(arr, n, i):

```
largest = i          #  Initialize largest as root

                     # i: The index of the root of the subtree being heapified.

left = 2 * i + 1     # Left child index

right = 2 * i + 2    #  Right child index


# If any child is larger than the root, update largest to that child's index.

# If the largest value is not the root, swap the root with the largest child.

#  If left_child is larger than the root

if left < n and arr[left] > arr[largest]:

    largest = left
```

 # If right_child is larger than largest so far

if right < n and arr[right] > arr[largest]:

largest = right

 # If largest is not root

if largest != i:

swap(arr[i], arr[largest])

 # Recursively heapify the affected sub-tree

heapify(arr, n, largest)

```python
def heapify(arr, n, i):
    largest = i  # Initialize largest as root
    left = 2 * i + 1  # Left child index
    right = 2 * i + 2  # Right child index

    # If left child exists and is greater than root
    if left < n and arr[left] > arr[largest]:
        largest = left

    # If right child exists and is greater than largest so far
    if right < n and arr[right] > arr[largest]:
        largest = right

    # If largest is not root
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]  # Swap

        # Recursively heapify the affected subtree
        heapify(arr, n, largest)
```

```python
def heap_sort(arr):
    n = len(arr)

    # Build a max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # One by one extract elements
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]  # Swap
        heapify(arr, i, 0)
```

COUNTING SORT:

```
CountingSort(arr, k):
  # arr is array to be sorted.
   # k: range of input values (0 to  k−1) ; k is  the maximum element.
  n = len(arr)
  # Create a count array to store the count of each unique object
  count = array of size k initialized to 0
  # Step 1: Store the count of each element
  for i = 0 to n - 1:
     count[arr[i]] += 1
  # Step 2: Modify the count array such that each element at index i contains the sum of previous counts. It helps determine the position of each element in the sorted output.
  for i = 1 to k - 1:
     count[i] += count[i - 1]
  # Step 3: Output each object from the input array into the sorted output array
  output = array of size n
  for i = n - 1 downto 0:
     output[count[arr[i]] - 1] = arr[i]
     count[arr[i]] -= 1
  # Step 4: Copy the sorted elements into original array
  for i = 0 to n - 1:
     arr[i] = output[i]
```

```python
def counting_sort(arr):
    # Find the maximum element in the array to determine the range of the cc
    max_val = max(arr)
    # Initialize count array with zeros
    count = [0] * (max_val + 1)
    # Initialize the output array
    output = [0] * len(arr)

    # Step 1: Store the count of each element
    for num in arr:
        count[num] += 1

    # Step 2: Modify the count array so that each element at index i contair
    for i in range(1, len(count)):
        count[i] += count[i - 1]

    # Step 3: Build the output array
    for num in reversed(arr):
        output[count[num] - 1] = num
        count[num] -= 1

    # Step 4: Copy the sorted elements back into the original array
    for i in range(len(arr)):
        arr[i] = output[i]
```

RADIX SORT:

It assumes that sorting numbers digit by digit will eventually result in a fully sorted list. Radix Sort can be performed using different variations, such as ***Least Significant Digit (LSD) Radix Sort or Most Significant Digit (MSD) Radix Sort***. *We use a stable sorting technique, such as* ***counting sort****, to sort the digits at each significant place*

```
RadixSort(arr, d):
  # d is the number of digits in the largest number
  for i = 1 to d:
    # Perform counting sort on the ith digit
    CountingSortByDigit(arr, i)

CountingSortByDigit(arr, digit):
  n = len(arr)
  output = array of size n
  count = array of size 10 initialized to 0

# Store the count of occurrences of each digit
  for i = 0 to n - 1:
    digit_of_element = (arr[i] // 10^(digit - 1)) % 10
    count[digit_of_element] += 1

  # Modify the count array to store the actual positions
  for i = 1 to 9:
    count[i] += count[i - 1]

  # Build the output array
  for i = n - 1 downto 0:
    digit_of_element = (arr[i] // 10^(digit - 1)) % 10
    output[count[digit_of_element] - 1] = arr[i]
    count[digit_of_element] -= 1
```

 # Copy the sorted elements back into the original array

for i = 0 to n - 1:

arr[i] = output[i]

```
def counting_sort(arr, exp):
    n = len(arr)
    output = [0] * n  # Output array
    count = [0] * 10  # Count array for digits (0 to 9)

    # Store count of occurrences of each digit
    for i in range(n):
        index = (arr[i] // exp) % 10
        count[index] += 1

    # Change count[i] so that it now contains the actual position of the dig
    for i in range(1, 10):
        count[i] += count[i - 1]

    # Build the output array
    for i in range(n - 1, -1, -1):
        index = (arr[i] // exp) % 10
        output[count[index] - 1] = arr[i]
        count[index] -= 1

    # Copy the output array to arr, so that arr contains sorted numbers acco
    for i in range(n):
        arr[i] = output[i]
```

```
def radix_sort(arr):
    # Find the maximum number to know the number of digits
    max_val = max(arr)

    # Do counting sort for every digit, exp is 10^i (i is the current digit
    exp = 1
    while max_val // exp > 0:
        counting_sort(arr, exp)
        exp *= 10
```

BUCKET SORT:

*The process of bucket sort can be understood as a **scatter-gather approach.***

- *First, partition the range into a fixed number of buckets.*
- *Then, toss every element into its appropriate bucket.*
- *After that, sort each bucket individually by applying a sorting algorithm.*
- *And at last, concatenate all the sorted buckets.*

BucketSort(arr):

```
n = len(arr)
# Step 1: Create n empty buckets
buckets = array of n empty lists
# Step 2: Put array elements into different buckets
for i = 0 to n - 1:
    bucket_index = floor(n * arr[i])  # Assuming arr[i] is in the range [0, 1)
    append arr[i] to buckets[bucket_index]
# Step 3: Sort individual buckets using a suitable sorting algorithm (e.g., Insertion Sort)
for i = 0 to n - 1:
    InsertionSort(buckets[i])
# Step 4: Concatenate all sorted buckets into the original array
index = 0
for i = 0 to n - 1:
    for j = 0 to length of buckets[i] - 1:
        arr[index] = buckets[i][j]
        index += 1
```

InsertionSort(bucket):

```
for i = 1 to length of bucket - 1:
    key = bucket[i]
    j = i - 1
```

 # Move elements of bucket[0..i-1] that are greater than key to one position ahead of their # # # current position

while j >= 0 and bucket[j] > key:

bucket[j + 1] = bucket[j]

j -= 1

bucket[j + 1] = key

```
import math

def insertion_sort(bucket):
    for i in range(1, len(bucket)):
        key = bucket[i]
        j = i - 1
        # Move elements of bucket[0..i-1] that are greater than key
        while j >= 0 and bucket[j] > key:
            bucket[j + 1] = bucket[j]
            j -= 1
        bucket[j + 1] = key
```

```
def bucket_sort(arr):
    n = len(arr)
    # Step 1: Create n empty buckets
    buckets = []
    for _ in range(n):
        buckets.append([])

    # Step 2: Put array elements into different buckets
    for i in range(n):
        bucket_index = math.floor(n * arr[i])  # Assuming arr[i] is in the
        buckets[bucket_index].append(arr[i])

    # Step 3: Sort individual buckets using a suitable sorting algorithm (e
    for i in range(n):
        insertion_sort(buckets[i])

    # Step 4: Concatenate all sorted buckets into the original array
    index = 0
    for i in range(n):
        for j in range(len(buckets[i])):
            arr[index] = buckets[i][j]
            index += 1
```

Other **SORTING ALGORITHMS :**

SHELL SORT (improves Insertion Sort).

COCKTAIL SORT (a bidirectional Bubble Sort)

COMB SORT (improves Bubble Sort)

HYBRID SORTING ALGORITHMS :

TIMSORT : combines Merge Sort and Insertion Sort *(implemented in built-in python sort())*

INTROSORT : combines Quick Sort, Heap Sort, and Insertion Sort *(in C++ standard library)*.

BOGO SORT (Permutation Sort, for fun) : randomly shuffles the array until it is sorted. The time complexity is O((n* n!). The algorithm successively generates permutations of its input until it finds one that is sorted. For example, if bogosort is used to sort a deck of cards, it would consist of checking if the deck were in order, and if it were not, one would throw the deck into the air, pick the cards up at random, and repeat the process until the deck is sorted.

BITONIC SORT: a PARALLEL SORTING ALGORITHM that is particularly suited for GPUs or multi-core processors. It is based on the concept of a **bitonic sequence**, which is a sequence of numbers that first increases and then decreases, or vice versa.

Bitonic Sort is used in settings where **parallelism is critical** because it divides the sorting process into **a fixed series of steps that can run simultaneously. Complexity : O(log n * log n)**.

Bitonic Sort works by recursively:

1. **Building Bitonic Sequences:** Transforming smaller sequences into bitonic sequences.
2. **Merging Bitonic Sequences:** Combining two adjacent bitonic sequences into a larger bitonic sequence.

The "Bitonic Merge" Operation

The core of the algorithm is **the "bitonic merge" operation**. It takes a bitonic sequence as input and transforms it into a fully sorted sequence. It does this by:

1. **Comparing and Swapping: Comparing elements that are n/2 positions apart** (where n is the length of the sequence) and swapping them if necessary to maintain a sorted order.
2. **Recursive Merging:** Recursively applying the bitonic merge to the two halves of the sequence.

```
BitonicSort(arr, low, size, direction):
  # Base case: If size is 1, the array is already sorted
  if size <= 1:
    return
  # Step 1: Divide the array into two halves
  mid = size // 2
  # Step 2: Sort the first half in ascending order
  BitonicSort(arr, low, mid, True)
  # Step 3: Sort the second half in descending order
  BitonicSort(arr, low + mid, mid, False)
  # Step 4: Merge the two halves into a bitonic sequence
  BitonicMerge(arr, low, size, direction)

BitonicMerge(arr, low, size, direction):
  # Base case: If size is 1, nothing to merge
  if size <= 1:
    return
  # Step 1: Compare and swap elements based on direction
  mid = size // 2
  for i = low to low + mid - 1:
    if direction == (arr[i] > arr[i + mid]):
      swap(arr[i], arr[i + mid])
  # Step 2: Recursively merge the two halves
  BitonicMerge(arr, low, mid, direction)
  BitonicMerge(arr, low + mid, mid, direction)


# Main Function
Sort(arr, size):
  BitonicSort(arr, 0, size, True)
# https://www.youtube.com/watch?v=KKuza40Mbwg
```

```python
def bitonic_merge(arr, low, cnt, direction):
    if cnt > 1:
        k = cnt // 2
        for i in range(low, low + k):
            if (direction == 1 and arr[i] > arr[i + k]) or (direction == 0 a
                arr[i], arr[i + k] = arr[i + k], arr[i]
        bitonic_merge(arr, low, k, direction)
        bitonic_merge(arr, low + k, k, direction)

def bitonic_sort_recursive(arr, low, cnt, direction):
    if cnt > 1:
        k = cnt // 2
        bitonic_sort_recursive(arr, low, k, 1)  # Ascending order
        bitonic_sort_recursive(arr, low + k, k, 0)  # Descending order
        bitonic_merge(arr, low, cnt, direction)

def bitonic_sort(arr, n, up):
    bitonic_sort_recursive(arr, 0, n, up)
```

```python
or (direction == 0 and arr[i] < arr[i + k]):
```
(on the row above)

Time Complexity of the Sorting Algorithms: (source : Copilot)

Sorting Algorithm	Best Case Time Complexity	Average Case Time Complexity	Worst Case Time Complexity	Space Complexity	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	O①	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	O①	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O①	No
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	O①	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$	Yes
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n)$	Yes
Bitonic Sort	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	No